KIT719 Project 2 — Architecture

Objective

A Q&A system for ICT occupations in Australia.

- **RAG** retrieves role descriptions, skills, and titles from OSCA_27_ICT.pdf with explicit **[Page N]** references.
- **Tool** calls the **Adzuna API** to obtain **median** salary (with sample count and source).
- **Router** decides whether to use **RAG / TOOL / BOTH**.

1) Overall System Design

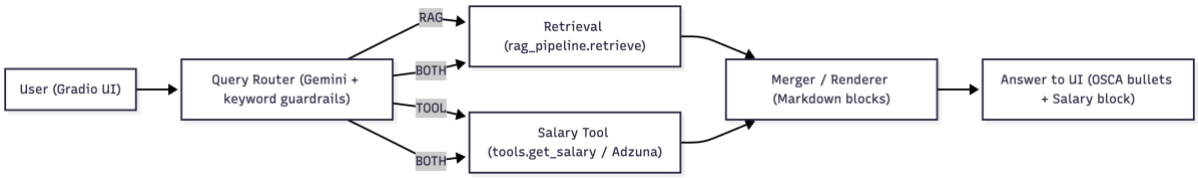

Figure 1: Overall Architecture Diagram.

Overall Flow (Description)

1) **User (Gradio UI)** submits a natural-language query.
2) **Query Router** classifies it with **Gemini** (if available) or **keyword guardrails** as fallback.
3) **RAG / BOTH** → retrieve top-k OSCA snippets and show them with **(Page N)**.
4) **TOOL / BOTH** → call **Adzuna**, filter outliers **30k–300k**, compute **median**; if city data is sparse, **fallback to Australia-wide median** (with a note).
5) **Merger / Renderer** combines the sections and returns a concise Markdown answer (**[Page N]** + **median**, **n samples**, **source**).

2) RAG Pipeline

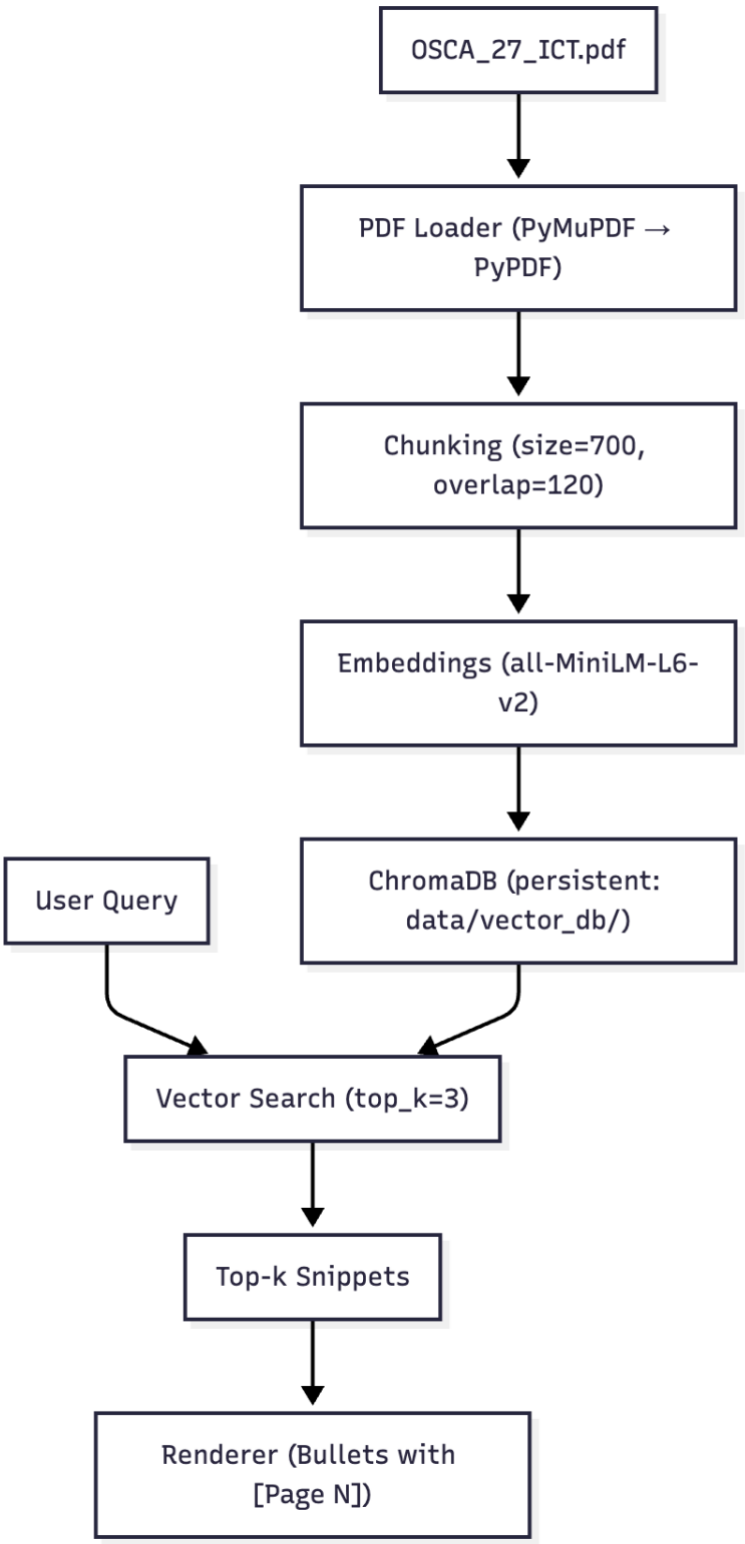

Figure 2: RAG Pipeline Diagram.

RAG Pipeline (Description)

3) **Loader:** prefer **PyMuPDF** (use_pymupdf: true); auto-fallback to **PyPDF** → robust text extraction.
4) **Chunking:** ≈ **700** chars with **120** overlap to preserve context.

5) **Embeddings: MiniLM-L6-v2** (Sentence-Transformers).
6) **Store: ChromaDB (persistent)** at data/vector_db/ (git-ignored).
7) **Search:** embed query → **vector search k=3** → top snippets.
8) **Renderer:** trim and display snippets with **(Page N)** for grounding.

Component ↔ File Mapping

Component	**Function**	**File**
UI / Orchestrator	Receive query, route, merge, render	main.py
Router	Classify **RAG / TOOL / BOTH** (Gemini + guardrails)	main.py, prompts.py
RAG	Build index + retrieve OSCA content	rag_pipeline.py
Salary Tool	Call Adzuna, filter outliers, compute median, fallback note	tools.py
Prompts / Templates	System prompt, router prompt, tool block	prompts.py
Config	RAG / Tool / LLM parameters and paths	config.yml